

◆ 1. Angular Routing – *E-Commerce Domain*

✚ Assignment: Build Product Navigation System

Questions

1. Create an Angular app for an **online shopping system** with the following routes:
 - /products
 - /products/:id
 - /cart
 - /checkout
 2. Implement **lazy loading** for the CheckoutModule.
 3. Add a **route guard**:
 - Only allow access to /checkout if user is logged in.
 4. Create a **wildcard route** for handling 404 pages.
 5. Implement **route parameters**:
 - Pass product ID from product list → product details page.
 6. Use **query parameters**:
 - Filter products (e.g., /products?category=electronics)
 7. Add **routerLinkActive** to highlight active navigation.
 8. Implement **nested routing**:
 - /products/details
 - /products/reviews
-

◆ 2. Angular Directives – *School Management Domain*

✚ Assignment: Student Dashboard UI

Questions

1. Use ***ngFor** to display a list of students:
 - Name, Class, Marks

2. Use ***ngIf**:
 - Show "Topper" badge if marks > 90
 3. Use **ngSwitch**:
 - Display grade based on marks:
 - A, B, C, Fail
 4. Apply **attribute directive** (ngClass):
 - Highlight failing students in red
 5. Apply **ngStyle**:
 - Dynamically change font size based on marks
 6. Create a **custom directive**:
 - appHighlight → highlights row on hover
 7. Create another directive:
 - Disable button if student is absent
-

3. Angular Pipes – *Banking Domain*

Assignment: Account Statement Viewer

Questions

1. Display transaction data using built-in pipes:
 - Currency Pipe (₹ format)
 - Date Pipe (dd-MM-yyyy)
2. Use **uppercase/lowercase pipe**:
 - Format account holder name
3. Implement **percent pipe**:
 - Show interest rate
4. Create a **custom pipe**:
 - Mask account number (e.g., XXXX1234)

5. Create another pipe:
 - Filter transactions above a certain amount
 6. Implement **chained pipes**:
 - Format + filter transactions
 7. Create a pipe:
 - Convert amount into words (bonus challenge)
-

◆ 4. Angular Services – *Healthcare Domain*

📌 Assignment: Patient Management System

Questions

1. Create a service PatientService:
 - Store patient list (id, name, age, disease)
2. Use **Dependency Injection**:
 - Inject service into multiple components
3. Implement CRUD operations:
 - Add patient
 - Update patient
 - Delete patient
 - Get all patients
4. Use **HttpClient**:
 - Fetch patient data from mock API (JSON server)
5. Implement **Observable pattern**:
 - Subscribe to patient data
6. Share data between components:
 - Use service as a shared state
7. Implement **error handling** in service:

- Handle API failures
8. Add a **loading indicator** using service
-

Advanced Challenge (Combine All Topics)

Domain: Food Delivery App

Build a mini app that includes:

- Routing:
 - /restaurants, /menu/:id, /orders
 - Directives:
 - Highlight best-rated restaurants
 - Pipes:
 - Format price, filter cuisine
 - Services:
 - Fetch restaurant data from API
-